JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface DoubleStream

All Superinterfaces:

AutoCloseable, BaseStream<Double, DoubleStream>

```
public interface DoubleStream
extends BaseStream<Double, DoubleStream>
```

A sequence of primitive double-valued elements supporting sequential and parallel aggregate operations. This is the `double` primitive specialization of `Stream`.

The following example illustrates an aggregate operation using `Stream` and `DoubleStream`, computing the sum of the weights of the red widgets:

```
    double sum = widgets.stream()
                        .filter(w -> w.getColor() == RED)
                        .mapToDouble(w -> w.getWeight())
                        .sum();
```

See the class documentation for `Stream` and the package documentation for java.util.stream for additional specification of streams, stream operations, stream pipelines, and parallelism.

Since:

1.8

See Also:

Stream, java.util.stream

Nested Class Summary

Nested Classes

Modifier and Type	Interface	Description
static interface	**DoubleStream.Builder**	A mutable builder for a DoubleStream.
static interface	**DoubleStream.DoubleMapMu**	Represents an operation that accepts a double-valued argument and a DoubleConsumer, and returns no result.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
boolean	**allMatch**(DoublePredicate predica	Returns whether all elements of this stream match the provided predicate.
boolean	**anyMatch**(DoublePredicate predica	Returns whether any elements of this stream match the provided predicate.
OptionalDouble	**average**()	Returns an OptionalDouble describing the arithmetic mean of elements of this stream, or an empty optional if this stream is empty.
Stream<Double>	**boxed**()	Returns a Stream consisting of the elements of this stream, boxed to Double.
static DoubleStream.Builder	**builder**()	Returns a builder for a DoubleStream.
<R> R	**collect**(Supplier<R> supplier, ObjDoubleConsumer<R> acc BiConsumer<R, R> combiner)	Performs a mutable reduction operation on the elements of this stream.
static DoubleStream	**concat**(DoubleStream a, DoubleStream b)	Creates a lazily concatenated stream whose elements are all the elements of the first stream followed by all the elements of the second stream.
long	**count**()	

		Returns the count of elements in this stream.
DoubleStream	**distinct**()	Returns a stream consisting of the distinct elements of this stream.
default DoubleStream	**dropWhile** (DoublePredicate predica	Returns, if this stream is ordered, a stream consisting of the remaining elements of this stream after dropping the longest prefix of elements that match the given predicate.
static DoubleStream	**empty**()	Returns an empty sequential DoubleStream.
DoubleStream	**filter** (DoublePredicate predica	Returns a stream consisting of the elements of this stream that match the given predicate.
OptionalDouble	**findAny**()	Returns an OptionalDouble describing some element of the stream, or an empty OptionalDouble if the stream is empty.
OptionalDouble	**findFirst**()	Returns an OptionalDouble describing the first element of this stream, or an empty OptionalDouble if the stream is empty.
DoubleStream	**flatMap** (DoubleFunction<? extends DoubleStream> mapper)	Returns a stream consisting of the results of replacing each element of this stream

		with the contents of a mapped stream produced by applying the provided mapping function to each element.
void	**forEach**(DoubleConsumer action)	Performs an action for each element of this stream.
void	**forEachOrdered**(DoubleConsumer action)	Performs an action for each element of this stream, guaranteeing that each element is processed in encounter order for streams that have a defined encounter order.
static DoubleStream	**generate**(DoubleSupplier s)	Returns an infinite sequential unordered stream where each element is generated by the provided DoubleSupplier.
static DoubleStream	**iterate**(double seed, DoublePredicate hasNext, DoubleUnaryOperator next	Returns a sequential ordered DoubleStream produced by iterative application of the given next function to an initial element, conditioned on satisfying the given hasNext predicate.
static DoubleStream	**iterate**(double seed, DoubleUnaryOperator f)	Returns an infinite sequential ordered DoubleStream produced by iterative application

		of a function `f` to an initial element `seed`, producing a `Stream` consisting of `seed`, `f(seed)`, `f(f(seed))`, etc.
DoubleStream	**limit**(long maxSize)	Returns a stream consisting of the elements of this stream, truncated to be no longer than `maxSize` in length.
DoubleStream	**map** (DoubleUnaryOperator map	Returns a stream consisting of the results of applying the given function to the elements of this stream.
default DoubleStream	**mapMulti** (DoubleStream.DoubleMapM	Returns a stream consisting of the results of replacing each element of this stream with multiple elements, specifically zero or more elements.
IntStream	**mapToInt** (DoubleToIntFunction map	Returns an `IntStream` consisting of the results of applying the given function to the elements of this stream.
LongStream	**mapToLong** (DoubleToLongFunction ma	Returns a `LongStream` consisting of the results of applying the given function to the elements of this stream.
<U> Stream<U>		Returns an object-valued `Stream`

	mapToObj(DoubleFunction<? extends U> mapper)	consisting of the results of applying the given function to the elements of this stream.
OptionalDouble	**max**()	Returns an `OptionalDouble` describing the maximum element of this stream, or an empty OptionalDouble if this stream is empty.
OptionalDouble	**min**()	Returns an `OptionalDouble` describing the minimum element of this stream, or an empty OptionalDouble if this stream is empty.
boolean	**noneMatch**(DoublePredicate predica	Returns whether no elements of this stream match the provided predicate.
static DoubleStream	**of**(double t)	Returns a sequential `DoubleStream` containing a single element.
static DoubleStream	**of**(double... values)	Returns a sequential ordered stream whose elements are the specified values.
DoubleStream	**peek**(DoubleConsumer action)	Returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed

		from the resulting stream.
double	**reduce**(double identity, DoubleBinaryOperator op)	Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value.
OptionalDouble	**reduce**(DoubleBinaryOperator op	Performs a reduction on the elements of this stream, using an associative accumulation function, and returns an OptionalDouble describing the reduced value, if any.
DoubleStream	**skip**(long n)	Returns a stream consisting of the remaining elements of this stream after discarding the first n elements of the stream.
DoubleStream	**sorted**()	Returns a stream consisting of the elements of this stream in sorted order.
double	**sum**()	Returns the sum of elements in this stream.
DoubleSummaryStatist	**summaryStatistics**()	Returns a DoubleSummaryStatistics describing various

		summary data about the elements of this stream.
default DoubleStream	**takeWhile**(DoublePredicate predica	Returns, if this stream is ordered, a stream consisting of the longest prefix of elements taken from this stream that match the given predicate.
double[]	**toArray**()	Returns an array containing the elements of this stream.

Methods declared in interface BaseStream

close, isParallel, iterator, onClose, parallel, sequential, spliterator, unordered

Method Details

filter

DoubleStream filter(DoublePredicate predicate)

Returns a stream consisting of the elements of this stream that match the given predicate.

This is an intermediate operation.

Parameters:

predicate - a non-interfering, stateless predicate to apply to each element to determine if it should be included

Returns:

the new stream

map

DoubleStream map(DoubleUnaryOperator mapper)

Returns a stream consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper` - a non-interfering, stateless function to apply to each element

Returns:

the new stream

mapToObj

`<U> Stream<U> mapToObj(DoubleFunction<? extends U> mapper)`

Returns an object-valued `Stream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Type Parameters:

`U` - the element type of the new stream

Parameters:

`mapper` - a non-interfering, stateless function to apply to each element

Returns:

the new stream

mapToInt

`IntStream mapToInt(DoubleToIntFunction mapper)`

Returns an `IntStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper` - a non-interfering, stateless function to apply to each element

Returns:

the new stream

mapToLong

`LongStream mapToLong(DoubleToLongFunction mapper)`

Returns a `LongStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper` - a non-interfering, stateless function to apply to each element

Returns:

the new stream

flatMap

```
DoubleStream flatMap(DoubleFunction<? extends DoubleStream> mapper)
```

Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element. Each mapped stream is `closed` after its contents have been placed into this stream. (If a mapped stream is `null` an empty stream is used, instead.)

This is an intermediate operation.

Parameters:

`mapper` - a non-interfering, stateless function to apply to each element which produces a `DoubleStream` of new values

Returns:

the new stream

See Also:

`Stream.flatMap(Function)`

mapMulti

```
default DoubleStream mapMulti
(DoubleStream.DoubleMapMultiConsumer mapper)
```

Returns a stream consisting of the results of replacing each element of this stream with multiple elements, specifically zero or more elements. Replacement is performed by applying the provided mapping function to each element in conjunction with a consumer argument that accepts replacement elements. The mapping function calls the consumer zero or more times to provide the replacement elements.

This is an intermediate operation.

If the consumer argument is used outside the scope of its application to the mapping function, the results are undefined.

Implementation Requirements:

The default implementation invokes `flatMap` on this stream, passing a function that behaves as follows. First, it calls the mapper function with a `DoubleConsumer` that accumulates replacement elements into a newly created internal buffer. When the mapper function returns, it creates a `DoubleStream` from the internal buffer. Finally, it returns this stream to `flatMap`.

Parameters:

`mapper` - a non-interfering, stateless function that generates replacement elements

Returns:

the new stream

Since:

16

See Also:

`Stream.mapMulti`

distinct

`DoubleStream distinct()`

Returns a stream consisting of the distinct elements of this stream. The elements are compared for equality according to `Double.compare(double, double)`.

This is a stateful intermediate operation.

Returns:

the result stream

sorted

`DoubleStream sorted()`

Returns a stream consisting of the elements of this stream in sorted order. The elements are compared for equality according to `Double.compare(double, double)`.

This is a stateful intermediate operation.

Returns:

the result stream

peek

`DoubleStream peek(DoubleConsumer action)`

Returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed from the resulting stream.

This is an intermediate operation.

For parallel stream pipelines, the action may be called at whatever time and in whatever thread the element is made available by the upstream operation. If the action modifies shared state, it is responsible for providing the required synchronization.

API Note:

This method exists mainly to support debugging, where you want to see the elements as they flow past a certain point in a pipeline:

```
DoubleStream.of(1, 2, 3, 4)
    .filter(e -> e > 2)
    .peek(e -> System.out.println("Filtered value: " + e))
    .map(e -> e * e)
    .peek(e -> System.out.println("Mapped value: " + e))
    .sum();
```

In cases where the stream implementation is able to optimize away the production of some or all the elements (such as with short-circuiting operations like `findFirst`, or in the example described in `count()`), the action will not be invoked for those elements.

Parameters:

`action` - a non-interfering action to perform on the elements as they are consumed from the stream

Returns:

the new stream

limit

`DoubleStream limit(long maxSize)`

Returns a stream consisting of the elements of this stream, truncated to be no longer than `maxSize` in length.

This is a short-circuiting stateful intermediate operation.

API Note:

While `limit()` is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of `maxSize`, since `limit(n)` is constrained to return not just

any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as `generate(DoubleSupplier)`) or removing the ordering constraint with `BaseStream.unordered()` may result in significant speedups of `limit()` in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with `limit()` in parallel pipelines, switching to sequential execution with `BaseStream.sequential()` may improve performance.

Parameters:

`maxSize` - the number of elements the stream should be limited to

Returns:

the new stream

Throws:

`IllegalArgumentException` - if `maxSize` is negative

skip

`DoubleStream skip(long n)`

Returns a stream consisting of the remaining elements of this stream after discarding the first `n` elements of the stream. If this stream contains fewer than `n` elements then an empty stream will be returned.

This is a stateful intermediate operation.

API Note:

While `skip()` is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of `n`, since `skip(n)` is constrained to skip not just any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as `generate(DoubleSupplier)`) or removing the ordering constraint with `BaseStream.unordered()` may result in significant speedups of `skip()` in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with `skip()` in parallel pipelines, switching to sequential execution with `BaseStream.sequential()` may improve performance.

Parameters:

`n` - the number of leading elements to skip

Returns:

the new stream

Throws:

`IllegalArgumentException` - if `n` is negative

takeWhile

`default DoubleStream takeWhile(DoublePredicate predicate)`

Returns, if this stream is ordered, a stream consisting of the longest prefix of elements taken from this stream that match the given predicate. Otherwise returns, if this stream is unordered, a stream consisting of a subset of elements taken from this stream that match the given predicate.

If this stream is ordered then the longest prefix is a contiguous sequence of elements of this stream that match the given predicate. The first element of the sequence is the first element of this stream, and the element immediately following the last element of the sequence does not match the given predicate.

If this stream is unordered, and some (but not all) elements of this stream match the given predicate, then the behavior of this operation is nondeterministic; it is free to take any subset of matching elements (which includes the empty set).

Independent of whether this stream is ordered or unordered if all elements of this stream match the given predicate then this operation takes all elements (the result is the same as the input), or if no elements of the stream match the given predicate then no elements are taken (the result is an empty stream).

This is a short-circuiting stateful intermediate operation.

API Note:

While `takeWhile()` is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, since the operation is constrained to return not just any valid prefix, but the longest prefix of elements in the encounter order. Using an unordered stream source (such as `generate(DoubleSupplier)`) or removing the ordering constraint with `BaseStream.unordered()` may result in significant speedups of `takeWhile()` in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with `takeWhile()` in parallel pipelines, switching to sequential execution with `BaseStream.sequential()` may improve performance.

Implementation Requirements:

The default implementation obtains the `spliterator` of this stream, wraps that spliterator so as to support the semantics of this operation on traversal, and returns a new stream associated with the wrapped spliterator. The returned stream preserves the execution characteristics of this stream (namely parallel or sequential execution as per `BaseStream.isParallel()`) but the wrapped spliterator may choose to not support splitting. When the

returned stream is closed, the close handlers for both the returned and this stream are invoked.

Parameters:

`predicate` - a non-interfering, stateless predicate to apply to elements to determine the longest prefix of elements.

Returns:

the new stream

Since:

9

dropWhile

```
default DoubleStream dropWhile(DoublePredicate predicate)
```

Returns, if this stream is ordered, a stream consisting of the remaining elements of this stream after dropping the longest prefix of elements that match the given predicate. Otherwise returns, if this stream is unordered, a stream consisting of the remaining elements of this stream after dropping a subset of elements that match the given predicate.

If this stream is ordered then the longest prefix is a contiguous sequence of elements of this stream that match the given predicate. The first element of the sequence is the first element of this stream, and the element immediately following the last element of the sequence does not match the given predicate.

If this stream is unordered, and some (but not all) elements of this stream match the given predicate, then the behavior of this operation is nondeterministic; it is free to drop any subset of matching elements (which includes the empty set).

Independent of whether this stream is ordered or unordered if all elements of this stream match the given predicate then this operation drops all elements (the result is an empty stream), or if no elements of the stream match the given predicate then no elements are dropped (the result is the same as the input).

This is a stateful intermediate operation.

API Note:

While `dropWhile()` is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, since the operation is constrained to return not just any valid prefix, but the longest prefix of elements in the encounter order. Using an unordered stream source (such as `generate(DoubleSupplier)`) or removing the ordering

constraint with BaseStream.unordered() may result in significant speedups of dropWhile() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with dropWhile() in parallel pipelines, switching to sequential execution with BaseStream.sequential() may improve performance.

Implementation Requirements:

The default implementation obtains the spliterator of this stream, wraps that spliterator so as to support the semantics of this operation on traversal, and returns a new stream associated with the wrapped spliterator. The returned stream preserves the execution characteristics of this stream (namely parallel or sequential execution as per BaseStream.isParallel()) but the wrapped spliterator may choose to not support splitting. When the returned stream is closed, the close handlers for both the returned and this stream are invoked.

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements to determine the longest prefix of elements.

Returns:

the new stream

Since:

9

forEach

```
void forEach(DoubleConsumer action)
```

Performs an action for each element of this stream.

This is a terminal operation.

For parallel stream pipelines, this operation does *not* guarantee to respect the encounter order of the stream, as doing so would sacrifice the benefit of parallelism. For any given element, the action may be performed at whatever time and in whatever thread the library chooses. If the action accesses shared state, it is responsible for providing the required synchronization.

Parameters:

action - a non-interfering action to perform on the elements

forEachOrdered

```
void forEachOrdered(DoubleConsumer action)
```

Performs an action for each element of this stream, guaranteeing that each element is processed in encounter order for streams that have a defined encounter order.

This is a terminal operation.

Parameters:

`action` - a non-interfering action to perform on the elements

See Also:

`forEach(DoubleConsumer)`

toArray

```
double[] toArray()
```

Returns an array containing the elements of this stream.

This is a terminal operation.

Returns:

an array containing the elements of this stream

reduce

```
double reduce(double identity,
              DoubleBinaryOperator op)
```

Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value. This is equivalent to:

```
    double result = identity;
    for (double element : this stream)
        result = accumulator.applyAsDouble(result, element)
    return result;
```

but is not constrained to execute sequentially.

The `identity` value must be an identity for the accumulator function. This means that for all `x`, `accumulator.apply(identity, x)` is equal to `x`. The `accumulator` function must be an associative function.

This is a terminal operation.

API Note:

Sum, min, max, and average are all special cases of reduction. Summing a stream of numbers can be expressed as:

```
    double sum = numbers.reduce(0, (a, b) -> a+b);
```

or more compactly:

```
    double sum = numbers.reduce(0, Double::sum);
```

While this may seem a more roundabout way to perform an aggregation compared to simply mutating a running total in a loop, reduction operations parallelize more gracefully, without needing additional synchronization and with greatly reduced risk of data races.

Parameters:

`identity` - the identity value for the accumulating function

`op` - an associative, non-interfering, stateless function for combining two values

Returns:

the result of the reduction

See Also:

`sum()`, `min()`, `max()`, `average()`

reduce

`OptionalDouble reduce(DoubleBinaryOperator op)`

Performs a reduction on the elements of this stream, using an associative accumulation function, and returns an `OptionalDouble` describing the reduced value, if any. This is equivalent to:

```
    boolean foundAny = false;
    double result = null;
    for (double element : this stream) {
        if (!foundAny) {
            foundAny = true;
            result = element;
        }
        else
            result = accumulator.applyAsDouble(result, element);
    }
    return foundAny ? OptionalDouble.of(result) : OptionalDouble.empty();
```

but is not constrained to execute sequentially.

The `accumulator` function must be an associative function.

This is a terminal operation.

Parameters:

`op` - an associative, non-interfering, stateless function for combining two values

Returns:

the result of the reduction

See Also:

`reduce(double, DoubleBinaryOperator)`

collect

```
<R> R collect(Supplier<R> supplier,
              ObjDoubleConsumer<R> accumulator,
              BiConsumer<R,R> combiner)
```

Performs a mutable reduction operation on the elements of this stream. A mutable reduction is one in which the reduced value is a mutable result container, such as an `ArrayList`, and elements are incorporated by updating the state of the result rather than by replacing the result. This produces a result equivalent to:

```
    R result = supplier.get();
    for (double element : this stream)
        accumulator.accept(result, element);
    return result;
```

Like `reduce(double, DoubleBinaryOperator)`, `collect` operations can be parallelized without requiring additional synchronization.

This is a terminal operation.

Type Parameters:

`R` - the type of the mutable result container

Parameters:

`supplier` - a function that creates a new mutable result container. For a parallel execution, this function may be called multiple times and must return a fresh value each time.

`accumulator` - an associative, non-interfering, stateless function that must fold an element into a result container.

`combiner` - an associative, non-interfering, stateless function that accepts two partial result containers and merges them, which must be compatible

with the accumulator function. The combiner function must fold the elements from the second result container into the first result container.

Returns:

the result of the reduction

See Also:

`Stream.collect(Supplier, BiConsumer, BiConsumer)`

sum

```
double sum()
```

Returns the sum of elements in this stream. Summation is a special case of a reduction. If floating-point summation were exact, this method would be equivalent to:

```
    return reduce(0, Double::sum);
```

However, since floating-point summation is not exact, the above code is not necessarily equivalent to the summation computation done by this method.

The value of a floating-point sum is a function both of the input values as well as the order of addition operations. The order of addition operations of this method is intentionally not defined to allow for implementation flexibility to improve the speed and accuracy of the computed result. In particular, this method may be implemented using compensated summation or other technique to reduce the error bound in the numerical sum compared to a simple summation of `double` values. Because of the unspecified order of operations and the possibility of using differing summation schemes, the output of this method may vary on the same input elements.

Various conditions can result in a non-finite sum being computed. This can occur even if the all the elements being summed are finite. If any element is non-finite, the sum will be non-finite:

- If any element is a NaN, then the final sum will be NaN.
- If the elements contain one or more infinities, the sum will be infinite or NaN.
    - If the elements contain infinities of opposite sign, the sum will be NaN.
    - If the elements contain infinities of one sign and an intermediate sum overflows to an infinity of the opposite sign, the sum may be NaN.

It is possible for intermediate sums of finite values to overflow into opposite-signed infinities; if that occurs, the final sum will be NaN even if the elements

are all finite. If all the elements are zero, the sign of zero is *not* guaranteed to be preserved in the final sum.

This is a terminal operation.

API Note:

Elements sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

the sum of elements in this stream

min

`OptionalDouble min()`

Returns an `OptionalDouble` describing the minimum element of this stream, or an empty OptionalDouble if this stream is empty. The minimum element will be `Double.NaN` if any stream element was NaN. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero. This is a special case of a reduction and is equivalent to:

```
    return reduce(Double::min);
```

This is a terminal operation.

Returns:

an `OptionalDouble` containing the minimum element of this stream, or an empty optional if the stream is empty

max

`OptionalDouble max()`

Returns an `OptionalDouble` describing the maximum element of this stream, or an empty OptionalDouble if this stream is empty. The maximum element will be `Double.NaN` if any stream element was NaN. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero. This is a special case of a reduction and is equivalent to:

```
    return reduce(Double::max);
```

This is a terminal operation.

Returns:

an `OptionalDouble` containing the maximum element of this stream, or an empty optional if the stream is empty

count

`long count()`

Returns the count of elements in this stream. This is a special case of a reduction and is equivalent to:

```
    return mapToLong(e -> 1L).sum();
```

This is a terminal operation.

API Note:

An implementation may choose to not execute the stream pipeline (either sequentially or in parallel) if it is capable of computing the count directly from the stream source. In such cases no source elements will be traversed and no intermediate operations will be evaluated. Behavioral parameters with side-effects, which are strongly discouraged except for harmless cases such as debugging, may be affected. For example, consider the following stream:

```
    DoubleStream s = DoubleStream.of(1, 2, 3, 4);
    long count = s.peek(System.out::println).count();
```

The number of elements covered by the stream source is known and the intermediate operation, `peek`, does not inject into or remove elements from the stream (as may be the case for `flatMap` or `filter` operations). Thus the count is 4 and there is no need to execute the pipeline and, as a side-effect, print out the elements.

Returns:

the count of elements in this stream

average

`OptionalDouble average()`

Returns an `OptionalDouble` describing the arithmetic mean of elements of this stream, or an empty optional if this stream is empty.

The computed average can vary numerically and have the special case behavior as computing the sum; see `sum()` for details.

The average is a special case of a reduction.

This is a terminal operation.

API Note:

Elements sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

an `OptionalDouble` containing the average element of this stream, or an empty optional if the stream is empty

summaryStatistics

`DoubleSummaryStatistics summaryStatistics()`

Returns a `DoubleSummaryStatistics` describing various summary data about the elements of this stream. This is a special case of a reduction.

This is a terminal operation.

Returns:

a `DoubleSummaryStatistics` describing various summary data about the elements of this stream

anyMatch

`boolean anyMatch(DoublePredicate predicate)`

Returns whether any elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then `false` is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *existential quantification* of the predicate over the elements of the stream (for some x P(x)).

Parameters:

`predicate` - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

`true` if any elements of the stream match the provided predicate, otherwise `false`

allMatch

`boolean allMatch(DoublePredicate predicate)`

Returns whether all elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then `true` is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the predicate over the elements of the stream (for all x P(x)). If the stream is empty, the quantification is said to be *vacuously satisfied* and is always `true` (regardless of P(x)).

Parameters:

`predicate` - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

`true` if either all elements of the stream match the provided predicate or the stream is empty, otherwise `false`

noneMatch

```
boolean noneMatch(DoublePredicate predicate)
```

Returns whether no elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then `true` is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the negated predicate over the elements of the stream (for all x ~P(x)). If the stream is empty, the quantification is said to be vacuously satisfied and is always `true`, regardless of P(x).

Parameters:

`predicate` - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

`true` if either no elements of the stream match the provided predicate or the stream is empty, otherwise `false`

findFirst

```
OptionalDouble findFirst()
```

Returns an `OptionalDouble` describing the first element of this stream, or an empty `OptionalDouble` if the stream is empty. If the stream has no encounter order, then any element may be returned.

This is a short-circuiting terminal operation.

Returns:

an `OptionalDouble` describing the first element of this stream, or an empty `OptionalDouble` if the stream is empty

findAny

```
OptionalDouble findAny()
```

Returns an `OptionalDouble` describing some element of the stream, or an empty `OptionalDouble` if the stream is empty.

This is a short-circuiting terminal operation.

The behavior of this operation is explicitly nondeterministic; it is free to select any element in the stream. This is to allow for maximal performance in parallel operations; the cost is that multiple invocations on the same source may not return the same result. (If a stable result is desired, use `findFirst()` instead.)

Returns:

an `OptionalDouble` describing some element of this stream, or an empty `OptionalDouble` if the stream is empty

See Also:

`findFirst()`

boxed

```
Stream<Double> boxed()
```

Returns a `Stream` consisting of the elements of this stream, boxed to `Double`.

This is an intermediate operation.

Returns:

a `Stream` consistent of the elements of this stream, each boxed to a `Double`

builder

```
static DoubleStream.Builder builder()
```

Returns a builder for a `DoubleStream`.

Returns:

a stream builder

empty

```
static DoubleStream empty()
```

Returns an empty sequential `DoubleStream`.

Returns:

an empty sequential stream

of

```
static DoubleStream of(double t)
```

Returns a sequential `DoubleStream` containing a single element.

Parameters:

`t` - the single element

Returns:

a singleton sequential stream

of

```
static DoubleStream of(double... values)
```

Returns a sequential ordered stream whose elements are the specified values.

Parameters:

`values` - the elements of the new stream

Returns:

the new stream

iterate

```
static DoubleStream iterate(double seed,
                            DoubleUnaryOperator f)
```

Returns an infinite sequential ordered `DoubleStream` produced by iterative application of a function `f` to an initial element `seed`, producing a `Stream` consisting of `seed`, `f(seed)`, `f(f(seed))`, etc.

The first element (position `0`) in the `DoubleStream` will be the provided `seed`. For `n > 0`, the element at position `n`,
will be the result of applying the function `f` to the element at position `n - 1`.

The action of applying `f` for one element *happens-before* the action of applying `f` for subsequent elements. For any given element the action may be performed in whatever thread the library chooses.

Parameters:

`seed` - the initial element

`f` - a function to be applied to the previous element to produce a new element

Returns:

a new sequential `DoubleStream`

iterate

```
static DoubleStream iterate(double seed,
                            DoublePredicate hasNext,
                            DoubleUnaryOperator next)
```

Returns a sequential ordered `DoubleStream` produced by iterative application of the given `next` function to an initial element, conditioned on satisfying the given `hasNext` predicate. The stream terminates as soon as the `hasNext` predicate returns false.

`DoubleStream.iterate` should produce the same sequence of elements as produced by the corresponding for-loop:

```
    for (double index=seed; hasNext.test(index); index = next.applyAsDouble(inde
        ...
    }
```

The resulting sequence may be empty if the `hasNext` predicate does not hold on the seed value. Otherwise the first element will be the supplied `seed` value, the next element (if present) will be the result of applying the `next` function to the `seed` value, and so on iteratively until the `hasNext` predicate indicates that the stream should terminate.

The action of applying the `hasNext` predicate to an element *happens-before* the action of applying the `next` function to that element. The action of applying the `next` function for one element *happens-before* the action of applying the `hasNext` predicate for subsequent elements. For any given element an action may be performed in whatever thread the library chooses.

Parameters:

`seed` - the initial element

`hasNext` - a predicate to apply to elements to determine when the stream must terminate.

`next` - a function to be applied to the previous element to produce a new element

Returns:

a new sequential `DoubleStream`

Since:

9

generate

```
static DoubleStream generate(DoubleSupplier s)
```

Returns an infinite sequential unordered stream where each element is generated by the provided `DoubleSupplier`. This is suitable for generating constant streams, streams of random elements, etc.

Parameters:

`s` - the `DoubleSupplier` for generated elements

Returns:

a new infinite sequential unordered `DoubleStream`

concat

```
static DoubleStream concat(DoubleStream a,
                           DoubleStream b)
```

Creates a lazily concatenated stream whose elements are all the elements of the first stream followed by all the elements of the second stream. The resulting stream is ordered if both of the input streams are ordered, and parallel if either of the input streams is parallel. When the resulting stream is closed, the close handlers for both input streams are invoked.

This method operates on the two input streams and binds each stream to its source. As a result subsequent modifications to an input stream source may not be reflected in the concatenated stream result.

API Note:

To preserve optimization opportunities this method binds each stream to its source and accepts only two streams as parameters. For example, the exact size of the concatenated stream source can be computed if the exact size of each input stream source is known. To concatenate more streams without binding, or without nested calls to this method, try creating a stream of streams and flat-mapping with the identity function, for example:

```
    DoubleStream concat = Stream.of(s1, s2, s3, s4).flatMapToDouble(s -> s);
```

Implementation Note:

Use caution when constructing streams from repeated concatenation. Accessing an element of a deeply concatenated stream can result in deep call chains, or even `StackOverflowError`.

Parameters:

a - the first stream

b - the second stream

Returns:

the concatenation of the two input streams

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.